09 KUBERNETES

Module 9: RBAC & Access Control

Who can do what, where — and how to prove it

ROLE | BINDING | CLUSTERROLE | SERVICEACCOUNT | can-i

2026 training edition

Least privilege • namespace scope • cluster scope • workload identity

9 RECAP + AGENDA

From Runtime to Access Control

01 What the series has built so far

01	02	03	04	05	06	07	08
Docker	**Architecture**	**Pods / Services**	**Deployments**	**Operations**	**Config**	**Scheduling**	**Storage**
images • runtime	control plane • kubectl	lifecycle • networking	rollouts • labels	nodes • jobs	Secrets • sidecars	resources • probes	PV • PVC • state

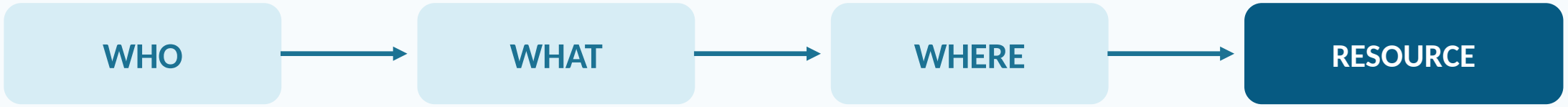
Now add the control plane for access.

A	**RBAC fundamentals**	who • role • binding • scope
B	**Roles & RoleBindings**	namespace-scoped permissions
C	**ClusterRoles & bindings**	cluster-wide permissions
D	**ServiceAccounts**	identity for Pods and automation
E	**Lab + 2026**	verify • troubleshoot • modernize

R RBAC FUNDAMENTALS

Why RBAC?

Authentication answers “who are you?” Authorization answers “what may you do?” RBAC is Kubernetes’ policy model for the second question.



Without RBAC

A credential with broad cluster access can perform far more actions than the job requires.

With RBAC

Grant the smallest useful set of verbs on the smallest useful resources.

Operational habit

When access fails, test the exact verb/resource/namespace with kubectl auth can-i.

least privilege | **explicit scope** | **verify**

5 RBAC FUNDAMENTALS

The Five Pillars of RBAC

WHO	ROLE	BIND	CLUSTER ROLE	CLUSTER BIND
Users / Groups / ServiceAccounts	**Role**	**RoleBinding**	**ClusterRole**	**ClusterRoleBinding**
Subjects	Namespace permissions	Attaches subject → Role	Cluster-wide / cluster-scoped	Attaches subject → ClusterRole

Mental model **Subject → Binding → Role → API permissions**

Role vs ClusterRole: The Scope Boundary

Namespace A

Role

Only this namespace

namespaced resources

Role → one namespace

ClusterRole

Can grant across namespaces

or govern cluster-scoped resources

ClusterRole → cluster scope

R

ROLES + ROLEBINDINGS

What Is a Role?

A Role is a named set of PolicyRules inside one namespace. It grants verbs on selected resources — nothing more.

role.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-reader
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list"]
```

apiGroups

Empty string means the core API group, where Pods live.

resources

The resource names the API object type being protected.

verbs

get/list allow read operations; create/update/delete are separate permissions.

default namespace only

Role YAML: Read Pods, Nothing More

role.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-reader
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list"]
```

- **apiVersion** — RBAC API version
- **kind** — Role object
- **metadata.namespace** — permission scope
- **rules** — the actual policy
- **verbs** — allowed actions

Rule of thumb

A Role defines permissions; it does not assign them to a person.

B ROLES + ROLEBINDINGS

RoleBinding: Attach the Permission to a Subject

A RoleBinding is the connector. It names the subject and references the Role; the binding itself lives in a namespace.




```
namespace: default
```

Subjects

User, Group, or ServiceAccount.

roleRef

Points to the Role that contains the rules.

Effect

The subject receives the Role's permissions in this namespace.

B ROLES + ROLEBINDINGS

RoleBinding YAML: Bind jai to pod-reader

rolebinding.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: default
subjects:
  - kind: User
    name: jai
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

subjects

Who receives the permission. This example binds the user “jai”.

roleRef

Which Role is being granted. The Role must exist in the same namespace.

Binding scope

Because the RoleBinding is in default, its effect is limited to default.

Role ≠ access

A ROLES + ROLEBINDINGS

Apply Role + RoleBinding

$ Terminal

```
kubectl apply -f role.yaml
kubectl apply -f rolebinding.yaml
```

Output

```
role.rbac.authorization.k8s.io/pod-reader
created
rolebinding.rbac.authorization.k8s.io/pod-
reader-binding created
```

Sequence

Create the Role first, then bind it. A Role without a binding does not grant access to jai.

ROLES + ROLEBINDINGS

Verify Permissions with kubectl auth can-i

Do not guess. Ask the authorization layer the exact question your workload will face.

$ Terminal

```
kubectl auth can-i list pods --as=jai
```

Output

```
yes
```

$ Terminal

```
kubectl auth can-i delete pods --as=jai
```

Output

```
no
```

allowed

denied

Key lesson

The Role grants get/list, so delete is denied. Literal yes/no output is the fastest RBAC troubleshooting signal.

Configure kubectl for a Real User Identity

Kubernetes signs jai's certificate itself through the CertificateSigningRequest API — nobody has to copy the cluster's CA private key.

$ Terminal

```
openssl genrsa -out jai.key 2048
openssl req -new -key jai.key -out jai.csr \
  -subj "/CN=jai/O=developers"
kubectl apply -f jai-csr.yaml   # request: <base64 of jai.csr>
kubectl certificate approve jai
kubectl get csr jai -o jsonpath='{.status.certificate}' \
  | base64 -d > jai.crt
kubectl config set-credentials jai --embed-certs=true \
  --client-certificate=jai.crt --client-key=jai.key
kubectl config set-context jai --cluster=kind-lab \
  --user=jai --namespace=default
kubectl --context jai auth whoami
```

Authentication

CN = the user name, O = a group. Kubernetes has no User objects — it trusts what the signed certificate says.

kubeconfig

A context = cluster + user + namespace. On kind the cluster is kind-lab (see kubectl config get-contexts).

Cloud note

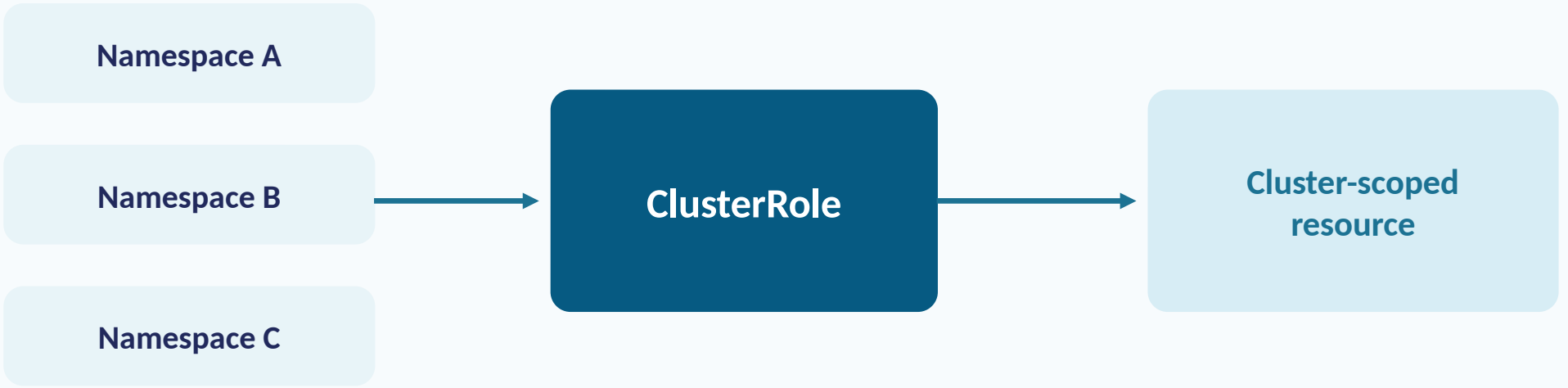
Managed services commonly integrate external IAM/OIDC instead of distributing the cluster CA private key.

C

CLUSTERROLES + BINDINGS

What Is a ClusterRole?

A ClusterRole is a reusable set of permissions that can apply cluster-wide, or to cluster-scoped resources such as Nodes and PersistentVolumes.



Examples

Pods across namespaces
Nodes
PersistentVolumes
other cluster resources

ClusterRole YAML: Read Pods Everywhere

clusterrole.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cluster-pod-reader
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list"]
```

No namespace field

A ClusterRole is cluster-scoped as an object.

Same PolicyRule model

apiGroups + resources + verbs are familiar from Role.

Reach

When bound cluster-wide, jai can list Pods in every namespace.

cluster scope

CB CLUSTERROLES + BINDINGS

ClusterRoleBinding: Bind Across the Cluster

The connector pattern is the same as RoleBinding — the scope changes.



Cluster-wide effect default dev prod ... all namespaces

Y CLUSTERROLES + BINDINGS

ClusterRoleBinding YAML

clusterrolebinding.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: cluster-pod-reader-binding
subjects:
  - kind: User
    name: jai
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: cluster-pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Subject

The same user jai can receive a different scope of access.

roleRef.kind

ClusterRole, not Role.

No namespace

ClusterRoleBinding is cluster-scoped and applies across namespaces.

CLUSTERROLES + BINDINGS

Apply + Verify Cluster-Wide Access

$ Terminal

```
kubectl auth can-i list pods -n kube-system \
  --as=jai
kubectl apply -f clusterrole.yaml \
  -f clusterrolebinding.yaml
kubectl auth can-i list pods -n kube-system \
  --as=jai
```

Output

```
no

clusterrole…/cluster-pod-reader created
clusterrolebinding…/cluster-pod-reader-
  binding created

yes
```

Test a place that was denied before

In default, jai could already list Pods — that proves nothing. kube-system went from no to yes: that's the ClusterRoleBinding working.

≠ CLUSTERROLES + BINDINGS

Role + RoleBinding vs ClusterRole + ClusterRoleBinding

Decision point	Role path	ClusterRole path
Scope	One namespace	Cluster-wide / cluster-scoped
Typical use	App-team access	Platform / cross-namespace access
Binding	RoleBinding	ClusterRoleBinding
Can protect Nodes?	No — Node is cluster-scoped	Yes, with appropriate rules
Can read Pods everywhere?	No	Yes, when bound cluster-wide

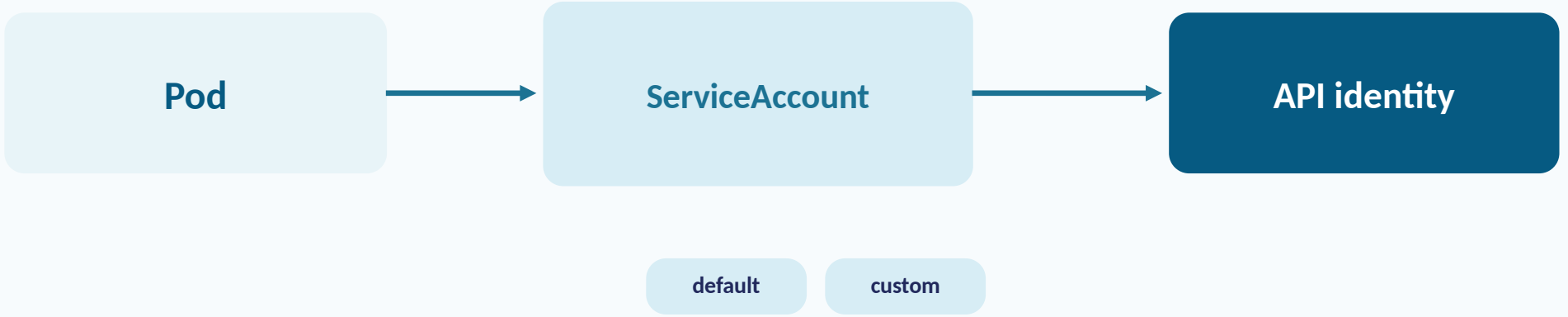
Least-privilege cue

Reach for the narrowest scope that satisfies the requirement.

SA SERVICEACCOUNTS

What Is a ServiceAccount?

A ServiceAccount is an identity for processes running in Pods — not a human user. Kubernetes creates a default ServiceAccount in every namespace.



Human user

External identity such as jai; represented to the API server by an auth mechanism.

Workload identity

ServiceAccount object lives in Kubernetes and can be bound with RBAC.

Token delivery

Modern Pods use short-lived projected ServiceAccount tokens by default.

API SERVICEACCOUNTS

Why ServiceAccounts Matter

01	**Controllers**	watch resources and reconcile state	least privilege
02	**Operators**	manage custom applications through the API	least privilege
03	**CI/CD**	deploy or inspect workloads from inside the cluster	least privilege
04	**Automation**	perform one narrowly-scoped API task	least privilege

Security rule

Never give a workload more API access than its job requires; avoid broad grants to the default ServiceAccount.

SA SERVICEACCOUNTS

Create + Bind a ServiceAccount

serviceaccount.yaml

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-reader
  namespace: default
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-reader-binding
  namespace: default
subjects:
  - kind: ServiceAccount
    name: app-reader
    namespace: default
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

$ Terminal

```
kubectl apply -f serviceaccount.yaml
kubectl get serviceaccounts
```

Output

```
NAME          AGE
app-reader    0s
default       11h
```

Modern token model

No token Secrets since v1.24, and on v1.37 kubectl no longer even shows a SECRETS column. Tokens are requested on demand.

Use a ServiceAccount in a Pod

app-reader-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: api-client
  namespace: default
spec:
  serviceAccountName: app-reader     # the Pod's identity when
it calls the API
  containers:
    - name: app
      image: nginx:1.28-alpine       # has curl, which we use
to call the API
      ports:
        - containerPort: 80
```

$ Terminal

```
kubectl apply -f app-reader-pod.yaml
kubectl describe pod api-client
```

Output

```
Service Account:  app-reader
Mounts:
  /var/run/secrets/kubernetes.io/
    serviceaccount (ro)
```

Token delivery

The kubelet asks for a ~1 h token (TokenRequest API), mounts it and renews it. It's never stored as a Secret.

SERVICEACCOUNTS

Verify a ServiceAccount's Permissions

$ Terminal

```
kubectl auth can-i list pods \
  --as=system:serviceaccount:default:app-reader
```

Output

```
yes
```

$ Terminal

```
kubectl auth can-i delete pods \
  --as=system:serviceaccount:default:app-reader
```

Output

```
no
```

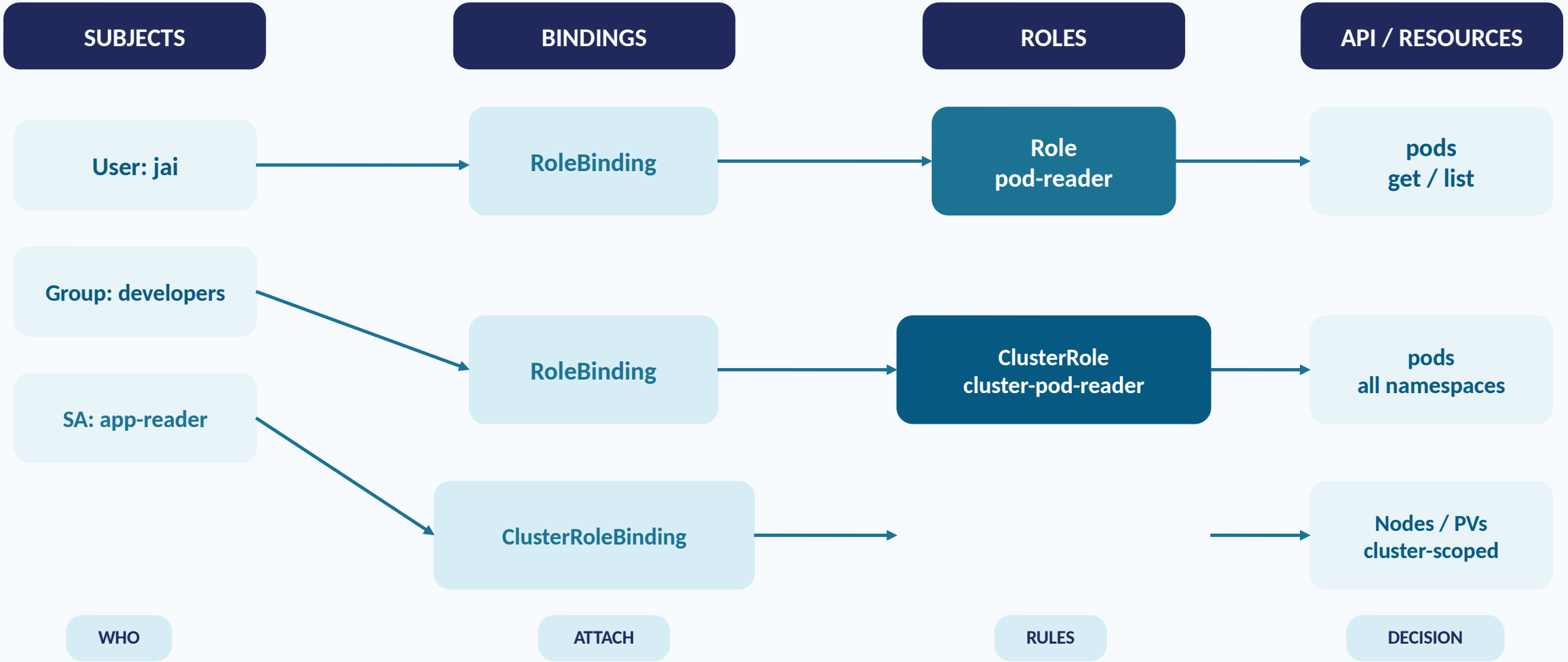
Impersonation syntax

system:serviceaccount:<namespace>:<name>. The command checks the exact authorization decision for that workload identity.

WRAP-UP

The Full RBAC Picture

One reference diagram: subjects receive bindings; bindings point to roles; roles contain PolicyRules; the API server authorizes the request.



LAB TRY IT YOURSELF

Hands-on Lab: Build and Prove Least Privilege

01

Create user jai

$ Terminal

```
kubectl certificate approve jai
```

02

Create Role

$ Terminal

```
kubectl apply -f role.yaml
```

03

Bind jai

$ Terminal

```
kubectl apply -f rolebinding.yaml
```

04

Prove yes/no

$ Terminal

```
kubectl auth can-i list pods --as=jai
kubectl auth can-i delete pods --as=jai
```

05

Expand scope

$ Terminal

```
kubectl apply -f clusterrole.yaml
kubectl apply -f clusterrolebinding.yaml
```

06

Prove the new reach

$ Terminal

```
kubectl auth can-i list pods -n kube-system --as=jai
```

Cleanup

Delete the dev and secure namespaces, the ClusterRoles, ClusterRoleBindings and the CSR — full tested walkthrough: demo.html Steps 2–11

LAB HANDS-ON LAB

Lab: A Real User, Signed by the Cluster

$ Terminal

```
openssl req -new -key jai.key -out jai.csr \
  -subj "/CN=jai/O=developers"
kubectl apply -f jai-csr.yaml
kubectl certificate approve jai
```

Output

```
jai   kube-apiserver-client   Pending
jai   …   Approved,Issued
subject=O = developers, CN = jai
```

$ Terminal

```
kubectl --context jai auth whoami
kubectl --context jai get pods
```

Output

```
Username   jai
Groups     [developers system:authenticated]

Forbidden: User "jai" cannot list
  resource "pods" … namespace "default"
```

Takeaway

Authentication says who you are; RBAC still says no until something is granted. demo.html Step 2

LAB HANDS-ON LAB

Lab: Role, RoleBinding and can-i

$ Terminal

```
kubectl apply -f role.yaml
kubectl --context jai get pods
kubectl apply -f rolebinding.yaml
```

Output

```
Forbidden          ← a Role alone does nothing
NAME    READY    STATUS
web     1/1      Running
```

$ Terminal

```
kubectl auth can-i get pods/log --as=jai
kubectl auth can-i get pods \
  --subresource=log --as=jai
```

Output

```
yes    ← checks a Pod NAMED "log"
no     ← the real question

kubectl logs web → Forbidden (pods/log)
```

Takeaway

Bind to grant. can-i answers exactly what you typed — confirm with a real command. Steps 3–4

LAB HANDS-ON LAB

Lab: Cluster Scope — ClusterRole and Nodes

$ Terminal

```
kubectl apply -f clusterrole.yaml \
  -f clusterrolebinding.yaml
kubectl auth can-i list pods \
  -n kube-system --as=jai
```

Output

```
before: no
after:  yes
jai now sees 15 Pods in all namespaces
```

$ Terminal

```
kubectl create role node-reader \
  --resource=nodes --verb=get,list
kubectl auth can-i list nodes --as=jai
kubectl --context jai get nodes
```

Output

```
Warning: resource 'nodes' is not namespace scoped
yes    ← misleading
Forbidden … at the cluster scope
ClusterRole node-reader → 3 nodes
```

Takeaway

Test a place that was denied before. Nodes, PVs and Namespaces need a ClusterRole. Steps 5–6

LAB HANDS-ON LAB

Lab: Teams and Escalation

$ Terminal

```
kubectl apply -f dev-team.yaml
# RoleBinding in dev:
#   Group developers → ClusterRole view
```

$ Terminal

```
kubectl --context jai -n dev create role \
  secret-reader --verb=get,list \
  --resource=secrets
```

Output

```
list deployments -n dev:   yes
list secrets -n dev:       no
create deployments -n dev: no
without --as-group:        no
```

Output

```
Forbidden: user "jai" … is attempting
to grant RBAC permissions not currently held:
{Resources:["secrets"], Verbs:["get" "list"]}
```

Takeaway

Reuse ClusterRoles per namespace, bind groups not people — and nobody can grant what they don't have. Steps 7–8

LAB HANDS-ON LAB

Lab: ServiceAccount Tokens in Action

$ Terminal

```
kubectl apply -f serviceaccount.yaml \
  -f app-reader-pod.yaml
kubectl get serviceaccounts
```

Output

```
NAME         AGE     ← no SECRETS column
app-reader   0s
expirationSeconds: 3607
sub: system:serviceaccount:default:app-reader
```

$ Terminal

```
curl --cacert $SA/ca.crt \
  -H "Authorization: Bearer $TOKEN" \
  $API/pods   (and DELETE $API/pods/web)
```

Output

```
list pods:     2 Pods
delete web:    403 Forbidden
list secrets:  403 Forbidden
automount false → no token folder
```

Takeaway

One ServiceAccount per app with only the rules it needs — and no token at all if it never calls the API. Steps 9–11

26 2026 MODERNIZATION

RBAC & Security in 2026: What's New

01	**Pod Security Admission**	Stable since v1.25. Namespace labels enforce Privileged / Baseline / Restricted Pod Security Standards.	**RBAC = who can do what; PSA = what Pods are allowed to do.**
02	**Fine-Grained Kubelet API Authorization**	Stable — verified GA on v1.37. Specific kubelet sub-resources replace broad nodes/proxy access.	**Better least privilege for monitoring / observability.**
03	**User Namespaces**	Stable — verified GA on v1.37. hostUsers: false maps root in the Pod to a nobody on the node (Module 3).	**A separate isolation layer — not a replacement for RBAC.**

2026 takeaway

Modern cluster security is layered: authentication → RBAC authorization → admission policy → runtime/container isolation.

Lab: Pod Security Admission

$ Terminal

```
kubectl apply -f secure-namespace.yaml
#   enforce: restricted
kubectl apply -f hostpath-in-secure.yaml
```

Output

```
Forbidden: violates PodSecurity
"restricted:latest": … restricted volume
types (volume "host" uses "hostPath") …
```

$ Terminal

```
kubectl apply -f restricted-pod.yaml
kubectl label --dry-run=server --overwrite \
  namespace default …enforce=restricted
```

Output

```
good-citizen   Running   uid=1000

Warning: existing pods in namespace
"default" violate … "restricted:latest"
```

Takeaway

RBAC decides who may create Pods; Pod Security decides what those Pods may do. Step 12

TROUBLESHOOTING

Common RBAC Pitfalls & Troubleshooting

01	**Role exists, access still denied**	Forgot the RoleBinding. A Role defines rules but does not assign them.
02	**Used Role for a cluster-scoped resource**	Nodes and PersistentVolumes are cluster-scoped; use an appropriate ClusterRole.
03	**“It should work” without testing**	Run kubectl auth can-i with the exact verb, resource, namespace, --as and --as-group — then confirm with a real command.
04	**Granted too much**	Avoid broad verbs/resources and avoid unnecessary cluster-wide bindings.
05	**ServiceAccount looks wrong**	Remember modern projected tokens may not appear as legacy Secret objects; inspect the Pod’s Service Account and projected mount.

RECAP

Module 9 Recap

- **Role** — namespace-scoped PolicyRules
- **RoleBinding** — attaches a Role to a subject
- **ClusterRole** — cluster-wide / cluster-scoped rules
- **ClusterRoleBinding** — attaches a ClusterRole cluster-wide
- **ServiceAccount** — workload identity for Pods
- **kubectl auth can-i** — fastest practical permission check

Core operational habit

Define the smallest permission → bind it to the right identity → verify with kubectl auth can-i → expand only when required.

09 KNOWLEDGE CHECK

9 questions — before you move on

Real access problems with the terminal output you'd see. The answer key is on the last quiz slide.

USERS | ROLES & BINDINGS | SERVICEACCOUNTS

Pause, choose, then check the key.

Module 9 quiz

Q1 KNOWLEDGE CHECK

Question 1 of 9 — Authentication vs RBAC

$ Scenario

```
kubectl --context maria auth whoami
kubectl --context maria get pods
```

Question

Maria's certificate works — whoami shows the name maria. But every command is Forbidden. What's missing?

Output

```
Username   maria
Groups     [qa system:authenticated]

Error from server (Forbidden): pods is
  forbidden: User "maria" cannot list …
```

Choose one

A. A new certificate
B. An RBAC binding: authentication worked, authorization didn't grant anything
C. A kubeconfig context
D. Approval of maria's CSR

Think about

Two different gates

Q2 KNOWLEDGE CHECK

Question 2 of 9 — Role without binding

$ Scenario

```
kubectl get role pod-reader -n shop
kubectl auth can-i list pods -n shop --as=maria
```

Question

The Role exists in shop, but maria still can't list Pods there. Fix?

Output

```
NAME         CREATED AT
pod-reader   2026-09-29T10:02:11Z

no
```

Choose one

A. Restart the API server
B. Add a deny rule for other users
C. Create a RoleBinding in shop that attaches pod-reader to maria
D. Change apiGroups to ["*"]

Think about

Rules vs assignment

Q3 KNOWLEDGE CHECK

Question 3 of 9 — Cluster-scoped resources

$ Scenario

```
kubectl auth can-i list nodes --as=ops
kubectl --context ops get nodes
```

Question

ops has a Role for nodes in default. can-i says yes, the real command is Forbidden. Why?

Output

```
Warning: resource 'nodes' is not namespace scoped
yes

Forbidden: … cannot list resource "nodes"
  … at the cluster scope
```

Choose one

A. Nodes are cluster-scoped: only a ClusterRole + ClusterRoleBinding can grant them
B. can-i is broken
C. The nodes are NotReady
D. ops needs --as-group

Think about

Where do Nodes live?

Q4 KNOWLEDGE CHECK

Question 4 of 9 — Proving a change

$ Scenario

```
kubectl apply -f clusterrolebinding.yaml
kubectl auth can-i list pods --as=jai
```

Question

A teammate says: "yes — so the ClusterRoleBinding works." jai already had a RoleBinding in default. What do you say?

Output

```
clusterrolebinding…/cluster-pod-reader-binding created

yes
```

Choose one

A. Agreed, yes is yes
B. Delete the RoleBinding first
C. Run can-i with --list
D. That proves nothing — test a namespace that was denied before, like kube-system

Think about

Before vs after

Q5 KNOWLEDGE CHECK

Question 5 of 9 — Team access

$ Scenario

```
# 20 developers, all in group "shop-devs"
# they need read-only access to namespace shop
```

Question

What's the cleanest way to give the whole team read-only access to namespace shop?

Output

```
kubectl get clusterrole view

NAME   CREATED AT
view   2026-09-29T04:43:11Z
```

Choose one

A. One RoleBinding in shop: Group shop-devs → ClusterRole view
B. 20 Roles and 20 RoleBindings
C. A ClusterRoleBinding to view
D. Share one certificate for the team

Think about

Groups and reusable ClusterRoles

Q6 KNOWLEDGE CHECK

Question 6 of 9 — Escalation

$ Scenario

```
kubectl --context jai -n dev create role \
  secret-reader --verb=get --resource=secrets
```

Question

jai is allowed to create Roles in dev. Why was this one refused?

Output

```
Forbidden: user "jai" … is attempting to grant
RBAC permissions not currently held:
{Resources:["secrets"], Verbs:["get"]}
```

Choose one

A. jai's certificate expired
B. Roles can't mention Secrets
C. RBAC won't let you grant permissions you don't hold yourself
D. dev has Pod Security enforced

Think about

Can you give away what you don't own?

Q7 KNOWLEDGE CHECK

Question 7 of 9 — ServiceAccounts

$ Scenario

```
# a controller in namespace ops must list Pods there
kubectl get pod reporter -o yaml | grep serviceAccount
```

Question

What's the right way to give this Pod API access?

Output

```
  serviceAccountName: default
```

Choose one

A. Bind the rules to the default ServiceAccount
B. Its own ServiceAccount + a Role + a RoleBinding, and set serviceAccountName
C. Mount an admin kubeconfig as a Secret
D. Use hostNetwork: true

Think about

Least privilege for workloads

Q8 KNOWLEDGE CHECK

Question 8 of 9 — Unneeded tokens

$ Scenario

```
kubectl exec web -- ls \
  /var/run/secrets/kubernetes.io/serviceaccount
```

Question

A web server never calls the Kubernetes API, but a security review flags the mounted token. Best fix?

Output

```
ca.crt
namespace
token
```

Choose one

A. Delete the token file at start-up
B. Make the token longer-lived
C. Rotate the ServiceAccount weekly
D. Set automountServiceAccountToken: false

Think about

No API calls, no token

Q9 KNOWLEDGE CHECK

Question 9 of 9 — Pod Security

$ Scenario

```
# jai has edit in dev and creates:
volumes: [{name: host, hostPath: {path: /}}]
```

Question

RBAC allowed jai to create this Pod. What stops it next time?

Output

```
pod/node-snooper created

# it can read every file on the node
```

Choose one

A. A ClusterRole for hostPath
B. Enforce Pod Security baseline or restricted on the dev namespace
C. Remove jai's edit role
D. Set readOnly: true

Think about

Who vs what

KEY KNOWLEDGE CHECK

Answer Key

Score yourself

```
8–9  Module 10 ready
6–7  re-read what you missed
≤ 5  redo Steps 2–12
```

Tip

```
Every answer names a slide
and a demo.html step.
```

Answers 1–5

Q1	B	bind something	→ slide 26 • Step 2
Q2	C	add a RoleBinding	→ slide 27 • Step 3
Q3	A	ClusterRole for nodes	→ slide 28 • Step 6
Q4	D	test a denied place	→ slide 28 • Step 5
Q5	A	group → view	→ slide 29 • Step 7

Answers 6–9

Q6	C	no escalation	→ slide 29 • Step 8
Q7	B	own ServiceAccount	→ slide 30 • Step 9
Q8	D	automount: false	→ slide 30 • Step 11
Q9	B	Pod Security	→ slide 32 • Step 12

Users & roles

Workloads

09 KUBERNETES

Next: Module 10 — Ingress & Beyond

Traffic entry, Gateway API, NetworkPolicy, Helm, CRDs & what's new

RBAC | ACCESS CONTROL | LEAST PRIVILEGE

You now have the authorization layer to secure the workloads you built in Modules 1–8.

Module 9 • complete